Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	Haze&Dust
Project / Solution Name	Vyte
Name (Leader)	Yau Zen Hao
Email & Phone (Leader)	Email : zenhao5514@gmail.com Phone : 010 863 3226
University Affiliation(s)	Tunku Abdul Rahman University of Management and Technology (TARUMT)
Live Prototype URL	https://hazendust.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/zenhao0915/Vyte

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

- User Interface: Built with React 19, TypeScript, Vite, Tailwind CSS and Lucide React (prototyped in Figma). The web dashboard collects customer orders (pasted WhatsApp text in multiple languages, photos or Excel files), supplier RFQ bids, invoice uploads, legacy database files via the Universal System Migrator, and compliance records, displaying prices in several currencies.

- Backend REST API: The UI sends JSON over HTTP to a Kotlin (JVM 21) REST API, which validates input and applies deterministic rules (supplier MOQ adherence, stock-shortage locks, shortage transfers, 30-day cash-flow runway, Malaysian RMCD SST tax codes (0% to 10%), and overdue QC task escalation payloads).

- AI: The backend calls the Google AI Studio Gemini API and a RunPod Serverless GPU endpoint for order parsing, invoice/shelf OCR, supplier defect-risk scoring, legacy schema mapping, quality-control suggestions and holding-cost analysis, returning structured JSON.

- Human Approval & SQLite: AI results wait in an approval queue for an Owner or Manager.(RBAC). Approved records are written to the SQLite, and every action is recorded in the audit log.

- Hosting: The application is developed and hosted on Replit; RunPod is used only for GPU inference. Live exchange rates are fetched from public forex APIs and WhatsApp Click-to-Chat dispatch .

(197 words)

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
TypeScript	Type-safe programming language used for building frontend UI components and backend logic	Apache 2.0
React	Frontend UI framework/library used for designing component-based user interfaces and interactive dashboards	MIT License
SQL	Relational database management system for persistent storage of inventory, order logs, and invoice data	Open Source / ISO Standard
Vite, Tailwind CSS And Lucide React	Frontend build tooling, responsive styling, spotlight tutorial, and UI iconography	MIT / ISC License
RunPod	HTTP client library used to communicate with RunPod Cloud GPU API endpoints for GenAI model inference	MIT License
Figma	UI/UX design tool used for prototyping dashboard layouts, modal designs, and user interaction flows	Proprietary Terms of Service
Replit	Cloud IDE development, build environment, and live web prototype deployment platform	Proprietary Terms of Service
Kotlin (JVM 21) & Gradle	Backend REST API server, deterministic validation, and webhook routing	Apache 2.0

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Google AI Studio	NLP order extraction (multi-language), invoice OCR and categorization, AI restock recommendations	Free
RunPod Inference Endpoints	Hosting and running GenAI models for real-time inference	Sponsor-provided
WhatsApp Business API	Sending payment reminders and AI-generated customer replies	Free
ExchangeRate-API (open.er-api.com)	Live MYR exchange rates for SGD, USD and RMB price conversion	Free
Frankfurter API (European Central Bank rates)	Backup source for live exchange rates	Free
Currency API by fawazahmed0 (via jsDelivr CDN)	Second backup source for live exchange rates	Free

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
RunPod	GPU compute for GenAI model inference and backend APIs	Sponsor provided
Replit	Development environment and live prototype hosting	Sponsor provided
SQLite (vyte.db)	Persistent storage for inventory, order, invoices	Managed
GitHub	Version Control, codebase hosting, and public review repository for hackathon judges	Free Public Repository

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	**Source / Provider**	**Usage (pre-trained / fine-tuned / self-trained)**	**License / Terms**
Gemini 2.0	Google AI Studio	Pre-trained	Google Terms
Claude Sonnet 5.5	Anthropic	Pre-trained	Anthropic Terms
Marvis	Tencent	Pre-trained	Tencent Terms

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	**Source / URL**	**Type (public / private / self-collected / synthetic)**	**License**	**Personal Data? (Y/N)**
Vyte Synthetic SME Operational Dataset	Self-generated by team (src/data/mockData.ts)	Synthetic	MIT	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Claude Sonnet 5.5 (web)	Writing React/TypeScript frontend components, Kotlin REST API backend logic, and structuring proposal documentation.	Moderate
Google AI Studio (web)	Prototyping prompts, testing NLP extraction logic, and generating API payloads.	Moderate
Gemini 4 Argon (web)	System architecture planning, SDG strategy alignment, and feature mapping.	Minor
Marvis by Tencent (Desktop app)	Code completion, inline debugging, and IDE workspace assistance.	Minor
NotebookLM (web)	Synthesizing hackathon guidelines and summarizing market competitor research.	Minor

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

All AI-generated code, architecture plans, and documentation were systematically reviewed, refactored, and tested by our team. Code snippets suggested by Claude Sonnet 5.5 and Marvis were manually validated in our IDE and tested against edge cases before committing to GitHub. Prompts tested in Google AI Studio and Gemini 4 Argon were tuned for accuracy. AI tools served to accelerate development, while our team retained full ownership and responsibility for the final application and data integrity.
(76 words)

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Cloud IDE & Deployment Platform	Used as the main development environment and for hosting the live prototype
RunPod	GPU Compute Instances	Used for running and hosting GenAI model inference (backend API calls)

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Yes, both Replit and RunPod were essential to the core functionality of the prototype. Replit was used as the development environment and for hosting the live web prototype, while RunPod provided the GPU compute needed for real-time GenAI inference. These platforms could not be easily replaced during the 1-week hackathon without significantly affecting development speed and performance. Without these resources, AI-driven features such as multi-language order parsing and restock recommendations would not be feasible in the current time frame.

(79 words)

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

In compliance with Malaysia's Personal Data Protection Act 2010, our system collects business inventory records, supplier contact numbers, customer order messages, and supplier invoices to process platform features efficiently and accurately.

Data Transmission & AI APIs:
Order text messages and invoice documents are processed via backend API routes and sent securely through encrypted HTTPS calls to our RunPod Cloud GPU instances and GenAI models for NLP parsing and document categorization. No personal user data is sold or used for model training.

Data Protection:
All persistent operational data, inventory logs, Document Vault contracts, and invoice entries are encrypted at rest within our secure SQL Database and protected by Role-Based Access Control (Owner, Manager, Ops Staff). Decommissioned SKUs are archived without deleting historical tax records.

Data Retention:
Operational records are retained only for the duration of the active business subscription. Temporary payload data sent during AI inference execution is deleted immediately after processing.

(150 words)

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

Key technical limitations and implemented mitigations include:

- AI Hallucinations & Formatting Errors:

  GenAI models can misparse complex order messages or misread raw invoice formatting.

  - Mitigation: We enforced strict Human-in-the-Loop Owner/Manager RBAC validation and Shadow/Copilot migration previews before any NLP-parsed order, OCR receipt, or legacy import updates the database or triggers invoice creation.

- Provider Dependency & Cloud Rate Limits:

  Heavy reliance on RunPod infrastructure exposes the platform to API rate limits or potential service downtime.
  - Mitigation: Implemented asynchronous API retry mechanisms, 3-provider forex failover (ExchangeRate-API → Frankfurter → CDN Forex), and a local Debug Mock Mode fallback so the workspace remains 100% operational during network latency.

- Database Latency:

  Large-scale real-time SQL queries during peak reorder cycles can slow down dashboard reporting.
  - Mitigation: Structured relational database indexing on high-frequency inventory tables.

(129 words)

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



24-10-2026